第二讲

- **上下文无关文法**
  - 形式定义
  - 归约和派生
  - 最左派生和最右派生
  - 文法的语言
- **语法分析树**

预备知识、记号

- ✧ 字母表
- ✧ 字符串
- ✧ 关于字符串的运算
- ✧ 字母表上的运算
- ✧ 语言
- ✧ 关于语言的运算

例子

✧ 以下哪些句子是正确的？

(a) The boy saw a girl with a telescope

(b) The a telescope boy with saw girl a

✧ 如何让计算机知道哪些句子是正确的？

观察规律

✧人说的话是有规律的

(a) The boy saw a girl with a telescope

(b) The professor gave a talk at the meeting

(c) A boy played a ball in the park

总结规律

✧可否为计算机总结语言的规律?

DT → the | a
NN → boy | girl | telescope | talk |
meeting | ball | park
VBD → saw | gave | played
IN → with | at | in

符号	含义
DT	冠词
NN	名词
VBD	动词
IN	介词

总结规律

✧可否为计算机总结语言的规律?

NP → DT NN

符号	含义
DT	冠词
NN	名词
VBD	动词
IN	介词
NP	名词短语

示例

- the boy
- a girl
- a telescope
- the professor
- the meeting

总结规律

✧可否为计算机总结语言的规律?

NP → DT NN
PP → IN NP

符号	含义
DT	冠词
NN	名词
VBD	动词
IN	介词
NP	名词短语
PP	介词短语

示例

- with a telescope
- at the meeting
- in the park

总结规律

✧可否为计算机总结语言的规律?

NP → DT NN
PP → IN NP
VP → VBD NP

示例

- saw a girl
- gave a talk
- played a ball

符号	含义
DT	冠词
NN	名词
VBD	动词
IN	介词
NP	名词短语
PP	介词短语
VP	动词短语

总结规律

✧可否为计算机总结语言的规律?

NP → DT NN
PP → IN NP
VP → VBD NP
VP → VP PP

符号	含义
DT	冠词
NN	名词
VBD	动词
IN	介词
NP	名词短语
PP	介词短语
VP	动词短语

示例

- saw a girl with a telescope
- gave a talk at the meeting

总结规律

✧可否为计算机总结语言的规律?

NP → DT NN
PP → IN NP
VP → VBD NP
VP → VP PP
S → NP VP

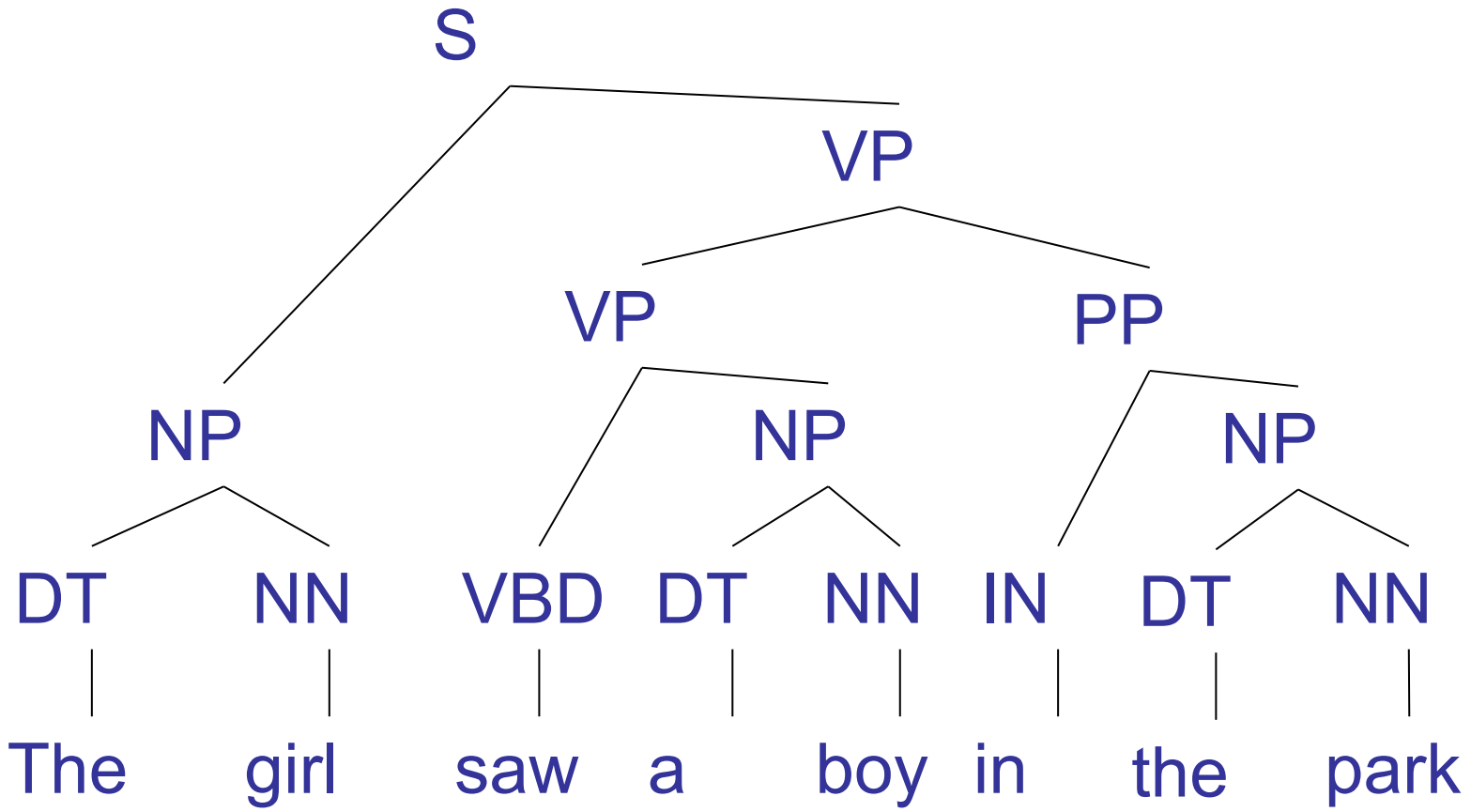
符号	含义
DT	冠词
NN	名词
VBD	动词
IN	介词
NP	名词短语
PP	介词短语
VP	动词短语
S	句子

示例

- The boy saw a girl with a telescope

应用规律

✧如何让计算机判断一个句子是否合法？



应用规律

✧如何让计算机判断一个句子是否合法？

反例：无法生成分析树（编程语法错误）

The a telescope boy with saw girl a

挑战：可以生成分析树但不合逻辑（编程逻辑错误）

The telescope saw a girl with a boy

字 母 表 （Alphabet）

- ✧ 概念　　形式符号的非空有限集合
- ✧ 记号　　常用 Σ 表示
- ✧ 举例
  - 英文字母表 { *a*, *b*, …, *z*, *A*, *B* , …, *Z* }
  - 英文标点符号表 { , ; : . ? ! ’ ‘ “ ” ( ) [ ] – - … }
  - 汉字表 { …, 自, …, 动 , …, 机, … }
  - 化学元素表 { H, He, Li, …, Une }
  - Σ = { *a*, *n*, *y*, 任,意 }

字 符 串 （string）

✧ 概念　字母表 Σ 上的一个字符串（串），或称为字（*word*），为 Σ 中字符构成的一个有限序列。 空串（*empty string*），常用 ε 表示，不包含任何字符。

举例　设 $\Sigma = \{ a, b \}$，则 ε, *a*, *aaa*, *baba* 等都是串

✧ 字符串 *w* 的长度，记为 $|w|$，是包含在 *w* 中字符的个数

举例　$|\varepsilon| = 0$，　$|bbaba| = 5$

关 于 字 符 串 的 运 算

✧ 连接（*concatenation*）

设 x, y为串，且 $x = a_1a_2 \ldots a_m$, $y = b_1b_2 \ldots b_n$,
则 x 与 y 的连接

$$xy = a_1a_2 \ldots a_m b_1b_2 \ldots b_n$$

✧ 连接运算的性质

– $(xy)z = x(yz)$

– $\varepsilon x = x\varepsilon = x$

– $|xy| = |x| + |y|$

字母表上的运算

✧ 幂运算

设 Σ 为字母表，n 为任意自然数，定义

（1）$\Sigma^0 = \{ \varepsilon \}$

（2）设 $x \in \Sigma^{n-1}$，$a \in \Sigma$， 则 $ax \in \Sigma^n$

（3）Σ^n 中的元素只能由（1）和（2）生成

举例 设 $\Sigma = \{ 0, 1 \}$，则

$\Sigma^0 = \{ \varepsilon \}$， $\Sigma^1 = \{ 0, 1 \}$，

$\Sigma^2 = \{ 00, 01 , 10 , 11 \}$， …

字母表上的运算

- ✧ $*$ 闭包　$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \ldots$
- ✧ $+$ 闭包　$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \ldots$
- ✧ $\Sigma^* = \Sigma^+ \cup \{ \varepsilon \}$
- ✧ 举例　设 $\Sigma = \{ 0, 1 \}$，则

$\Sigma^+ = \{ 0, 1, 00, 01, 10, 11, \ldots \}$

$\Sigma^* = \{ \varepsilon, 0, 1, 00, 01, 10, 11, \ldots \}$

语 言

✧ 概念 设 Σ 为字母表，则任何集合 $L \subseteq \Sigma^*$ 是字母表 Σ上的一个语言（*language*）

✧ 举例

– 英文单词集 {…, *English*, …, *words* , … }

– *C++* 语言程序集 { … }

– 汉语四字成语集 { …, 语不惊人, … , 言之有物, … }

– 化学分子式集{ …, H_2O, …, *NaCl* , … }

– {*any*, 任意}

✧ 比较 空语言 ϕ 与仅含空字的语言 { ε }

关于语言的运算

✧ 两个语言 L 和 M 的并（union）

$$L \cup M = \{ w \mid w \in L \vee w \in M \}$$

✧ 举例

设 $L = \{ 001,10,111 \}$, $M = \{\varepsilon, 001\}$, 则

$$L \cup M = \{ \varepsilon, 10, 001, 111 \}$$

关于语言的运算

✧ 两个语言 L 和 M 的连接（concatenation）

$$L \cdot M = \{w_1 w_2 \mid w_1 \in L \wedge w_2 \in M\}$$

✧ 通常记 $L \cdot M$ 为 LM

✧ 举例

设 $L = \{ 001,10,111 \}$, $M = \{\varepsilon , 001\}$,则

$LM = \{ 001, 10, 111, 001001, 10001, 111001 \}$

关于语言的运算

✧ 语言 L 的闭包（closure）

$L^* = L^0 \cup L^1 \cup L^2 \cup \ldots = \cup_{i \geq 0} L^i$，其中

$L^0 = \{\varepsilon\}$，　$L^1 = L$, $L^2 = LL$, …

$L^n = L^{n-1}L$

✧ 举例

设 $L = \{ 0, 11 \}$, 则

$L^* = \{ \varepsilon, 0, 11, 00, 011, 110, 1111, 000, 0011,$
$0110, 01111, 1100, 11011, 11110, 111111, \ldots \}$

单选题 1分

设置

令字母表Σ={a,b}，则克林闭包Σ*中元素的长度为?

A 可能为0，或者有限长，或者无限长

B 只能是无限的

C 可能是有限的，也可能是无限的

D 只能是有限的

提交

单选题 1分

设置

任意有穷集合的克林闭包一定是无穷集合。

A 正确

B 错误

提交

单选题 1分

设置

集合的克林闭包与正闭包一定不相等

A 正确

B 错误

提交

主要内容

- **上下文无关文法**
  - 形式定义
  - 归约和派生
  - 最左派生和最右派生
  - 文法的语言
- **语法分析树**

上下文无关文法

- 为什么引入上下文无法文法CFG(context-free grammar)?
  - 拥有足够强的表达能力，可以表示大多数程序设计语言的语法；实际上，几乎所有程序设计语言都是通过CFG来定义的。
  - 足够简单，我们可以构造有效的分析算法来检验一个给定字串是否是由某个CFG产生的(如LR 分析器和LL分析器)
  - 在程序设计语言的设计、编译器的实现等方面有重要应用，可应用在可扩展标记语言(XML)的格式类型定义(DTD)中等。

文法示例：自然语言的文法

- 简化的英语语法规则：

$$
\begin{aligned}
\langle \textit{sentence} \rangle &\rightarrow \langle \textit{noun-phrase} \rangle \langle \textit{verb-phrase} \rangle \\
\langle \textit{noun-phrase} \rangle &\rightarrow \langle \textit{article} \rangle \langle \textit{noun} \rangle \mid \langle \textit{article} \rangle \langle \textit{adjective} \rangle \langle \textit{noun} \rangle \\
\langle \textit{verb-phrase} \rangle &\rightarrow \langle \textit{verb} \rangle \mid \langle \textit{verb} \rangle \langle \textit{noun-phrase} \rangle \\
\langle \textit{article} \rangle &\rightarrow \texttt{a} \mid \texttt{the} \\
\langle \textit{noun} \rangle &\rightarrow \texttt{boy} \mid \texttt{girl} \mid \texttt{cat} \\
\langle \textit{adjective} \rangle &\rightarrow \texttt{big} \mid \texttt{small} \mid \texttt{blue} \\
\langle \textit{verb} \rangle &\rightarrow \texttt{sees} \mid \texttt{likes} \\
&\ldots
\end{aligned}
$$

自然语言的文法

- 使用定义的文法规则产生句子：

$$
\begin{aligned}
\langle sentence \rangle &\Rightarrow \langle noun\text{-}phrase \rangle \langle verb\text{-}phrase \rangle \\
&\Rightarrow \langle article \rangle \langle noun \rangle \langle verb\text{-}phrase \rangle \\
&\Rightarrow \langle article \rangle \langle noun \rangle \langle verb \rangle \langle noun\text{-}phrase \rangle \\
&\Rightarrow \langle article \rangle \langle noun \rangle \langle verb \rangle \langle article \rangle \langle adjective \rangle \langle noun \rangle \\
&\Rightarrow \texttt{the } \langle noun \rangle \langle verb \rangle \langle article \rangle \langle adjective \rangle \langle noun \rangle \\
&\Rightarrow \texttt{the girl } \langle verb \rangle \langle article \rangle \langle adjective \rangle \langle noun \rangle \\
&\Rightarrow \cdots \\
&\Rightarrow \texttt{the girl sees a blue cat}
\end{aligned}
$$

从sentence范畴到具体的一个句子，我们所使用的工具，就是文法。

回文与回文语言

- 定义:如果字符串$w \in \Sigma^*$满足$w = w^R$,则称字符串w为回文。

- 定义:如果语言L的字符串都是回文，则称L为回文语言。

$$L = \{w \in \Sigma^* | w = w^R\}$$

- 例:

$$\varepsilon, 010, 0000, radar, racecar, drawkward$$

上海自来水来自海上

一块五花肉花五块一

回文与回文语言

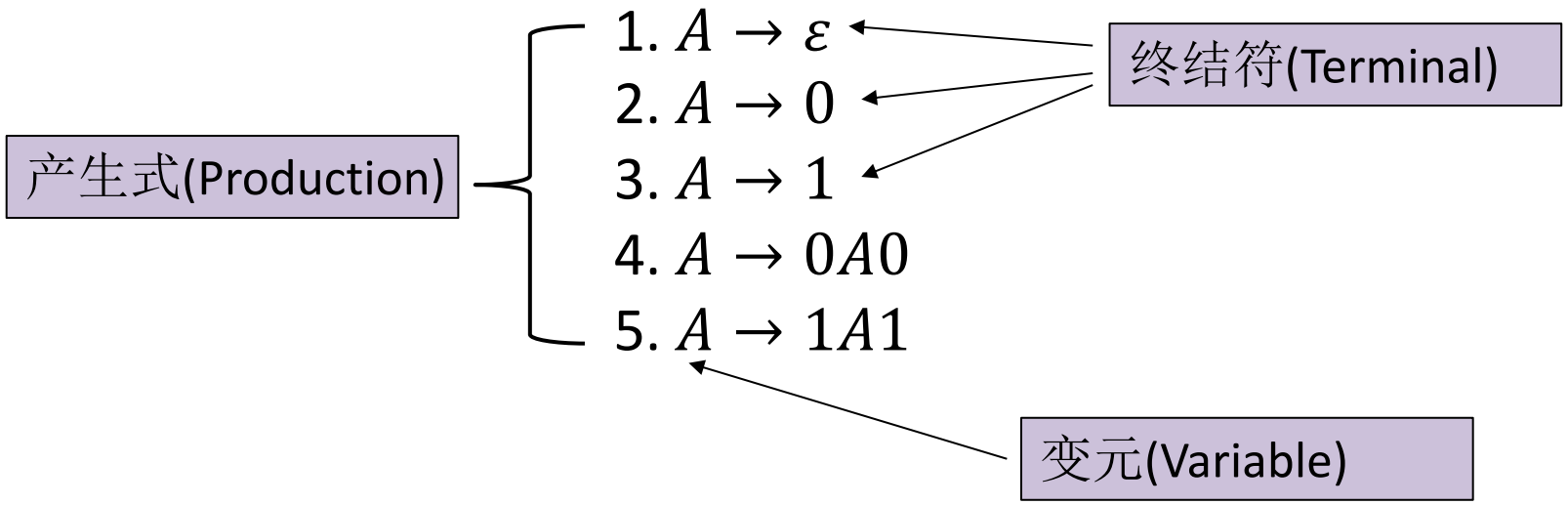
- **例：字母表$\Sigma = \{0,1\}$上的回文语言**

$$L_{pal} = \{w \in \{0,1\}^* | w = w^R\}$$

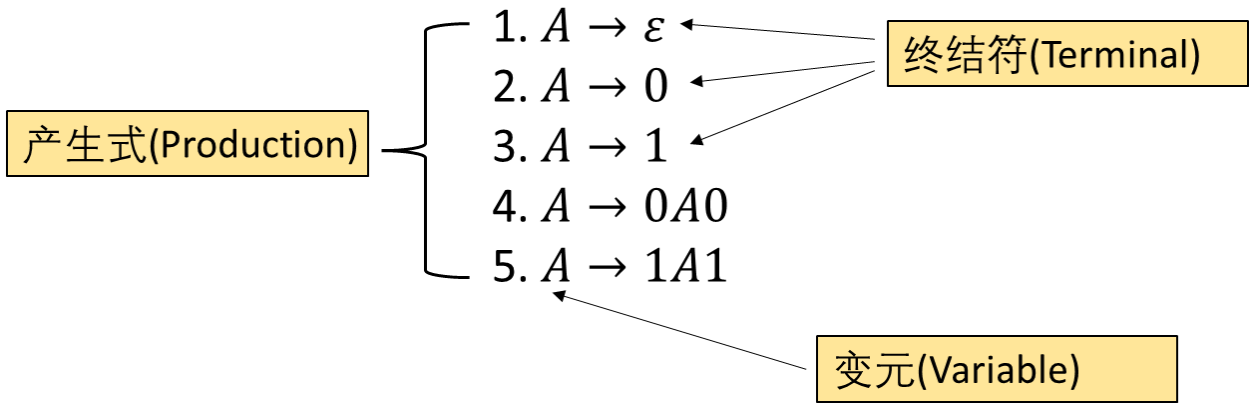
- **如何表示该语言呢？可使用递归的方式来定义：**
  - 首先，ε，0，1都是回文
  - 如果w是回文，$0w0$和$1w1$也是回文
- **使用嵌套定义表示这种递归结构**
  - 使用变元A表示回文语言中的字符串
  - $A \to \varepsilon, A \to 0, A \to 1$
  - $A \to 0A0$（仍为回文），$A \to 1A1$(仍为回文)。

回文与回文语言

- 回文语言文法



回文与回文语言



- **例:使用文法的嵌套定义表示一个字符串0110**
  - 利用$A \to 0A0$，那么有$A = 0A0$
  - 利用$A \to 1A1$，那么有$A = 01A10$
  - 利用$A \to \varepsilon$，那么有$A = 01\varepsilon 10 = 0110$

过程:

1. 选取能够产生符号串的一系列产生式
2. 在每一步，用某个产生式替换某个变元

上下文无关法的形式定义

- **定义：上下文无关文法（CFG, Context-Free Grammar，简称文法）G 是一个四元组**

$$G = (V, T, P, S)$$

- **其中，**
  - V:变元的有穷集, 变元也称为非终结符或语法范畴
  - T:终结符的有穷集（即字母表），且$V \cap T = \emptyset$
  - P:产生式的有穷集，每个产生式包括:
    - 一个变元，称为产生式的头(head)或左部;
    - 一个产生式符号→，读作定义为;
    - 一个$(V \cup T)^*$中的符号串, 称为体(body)或右部（由变元和终结符组成的字符串）
  - S:初始符号$S \in V$，表示文法开始的地方

上下文无关法的形式定义

- 产生式 $A \rightarrow \alpha$，读作 A 定义为α
- 如果有多个 A 的产生式:

$$A \rightarrow \alpha_1, A \rightarrow \alpha_2, \cdots, A \rightarrow \alpha_n$$

- 可简写为:

$$A \rightarrow \alpha_1 | \alpha_2 | \cdots | \alpha_n$$

- 文法中变元A的全体产生式，称为A产生式
- 续例：回文语言语法可以表示为？

$$G = (\{A\}, \{0,1\}, \{A \rightarrow \varepsilon | 0 | 1 | 0A0 | 1A1\}, A)$$

字符使用的一般约定

- 变元（非终结符）：$A, B, C, \ldots$
- 终结符：$0, 1, a, b, c, \ldots$
- 终结符或非终结符：$\ldots, X, Y, Z$
- 终结符构成的串：$\ldots, w, x, y, z$
- 终结符或非终结符组成的串：$\alpha, \beta, \gamma, \ldots$

文法示例:简化的算数表达式

- **两个变元:**
  - 带有“加”和“乘”$(+,*)$的表达式本身
  - 标识符:以 $\{a,b\}$ 开头,由$\{a,b,0,1\}$ 组成的字符串
- **用文法G_{exp}表示，** $G_{exp}=(\{E,I\},\{a,b,0,1,+,*,(,)\},P,E)$, **其中产生式集$P$中有10条产生式。**

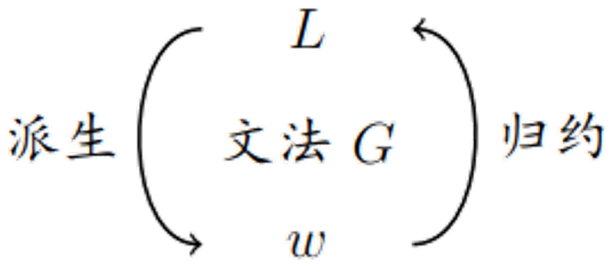
1. $E \to I$
2. $E \to E+E$
3. $E \to E*E$
4. $E \to (E)$
5. $I \to a$
6. $I \to b$
7. $I \to Ia$
8. $I \to Ib$
9. $I \to I0$
10. $I \to I1$

主要内容

- **上下文无关文法**
  - 形式定义
  - 归约和派生
  - 最左派生和最右派生
  - 文法的语言
- **语法分析树**
- **文法和语言的歧义性**
- **文法的化简与范式**

归约和派生

- **非形式定义：**
  - 从字符串到文法变元的分析过程，称为归约(reduction)；
  - 从文法变元到字符串的分析过程，称为派生(derivation)。



- **归约：自底向上，由产生式的体向头的分析**
- **派生：自顶向下，由产生式的头向体分析**

归约和派生

- 用算数表达式文法G_{exp}，将 $a * (a + b00)$ 归约的过程

1. $E \to I$
2. $E \to E + E$
3. $E \to E * E$
4. $E \to (E)$
5. $I \to a$
6. $I \to b$
7. $I \to Ia$
8. $I \to Ib$
9. $I \to I0$
10. $I \to I1$

$a * (a + b00)$:

利用(5)，(6)：$I * (I + I00)$

归约和派生

- 用算数表达式文法G_{exp}，将 $a * (a + b00)$ 归约的过程

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$
5. $I \rightarrow a$
6. $I \rightarrow b$
7. $I \rightarrow Ia$
8. $I \rightarrow Ib$
9. $I \rightarrow I0$
10. $I \rightarrow I1$

$a * (a + b00)$:

利用(5),(6): $I * (I + I00)$
利用(9): $I * (I + I0)$

归约和派生

- 用算数表达式文法G_{exp}，将 $a * (a + b00)$ 归约的过程

1. $E \to I$
2. $E \to E + E$
3. $E \to E * E$
4. $E \to (E)$
5. $I \to a$
6. $I \to b$
7. $I \to Ia$
8. $I \to Ib$
9. $I \to I0$
10. $I \to I1$

$a * (a + b00)$:

利用(5),(6): $I * (I + I00)$
利用(9): $I * (I + I0)$
利用(9): $I * (I + I)$

归约和派生

- 用算数表达式文法G_{exp}，将 $a * (a + b00)$ 归约的过程

1. $E \to I$
2. $E \to E + E$
3. $E \to E * E$
4. $E \to (E)$
5. $I \to a$
6. $I \to b$
7. $I \to Ia$
8. $I \to Ib$
9. $I \to I0$
10. $I \to I1$

$a * (a + b00)$:

利用(5),(6): $I * (I + I00)$
利用(9): $I * (I + I0)$
利用(9): $I * (I + I)$
利用(1): $E * (E + E)$

归约和派生

- 用算数表达式文法G_{exp}，将 $a * (a + b00)$ 归约的过程

1. $E \to I$
2. $E \to E + E$
3. $E \to E * E$
4. $E \to (E)$
5. $I \to a$
6. $I \to b$
7. $I \to Ia$
8. $I \to Ib$
9. $I \to I0$
10. $I \to I1$

$a * (a + b00)$:

利用(5),(6): $I * (I + I00)$
利用(9): $I * (I + I0)$
利用(9): $I * (I + I)$
利用(1): $E * (E + E)$
利用(2): $E * (E)$

归约和派生

- 用算数表达式文法G_{exp}，将 $a * (a + b00)$ 归约的过程

1. $E \to I$
2. $E \to E + E$
3. $E \to E * E$
4. $E \to (E)$
5. $I \to a$
6. $I \to b$
7. $I \to Ia$
8. $I \to Ib$
9. $I \to I0$
10. $I \to I1$

$a * (a + b00)$:

利用(5),(6): $I * (I + I00)$
利用(9): $I * (I + I0)$
利用(9): $I * (I + I)$
利用(1): $E * (E + E)$
利用(2): $E * (E)$
利用(4): $E * E$

归约和派生

- 用算数表达式文法G_{exp}，将 $a * (a + b00)$ 归约的过程

1. $E \to I$
2. $E \to E + E$
3. $E \to E * E$
4. $E \to (E)$
5. $I \to a$
6. $I \to b$
7. $I \to Ia$
8. $I \to Ib$
9. $I \to I0$
10. $I \to I1$

$a * (a + b00)$:

利用(5),(6): $I * (I + I00)$
利用(9): $I * (I + I0)$
利用(9): $I * (I + I)$
利用(1): $E * (E + E)$
利用(2): $E * (E)$
利用(4): $E * E$
利用(3): E

可以归约到文法的开始符号E。

多选题 对于CFG $G_{exp} = (\{E,O\}, \{ (,), +, *, v, d \}, P, E)$

以下哪些字符串可以归约为E?

P为
（1）$E \rightarrow EOE$
（2）$E \rightarrow (E)$
（3）$E \rightarrow v$
（4）$E \rightarrow d$
（5）$O \rightarrow +$
（6）$O \rightarrow *$

A $(v + d) * (d + d)$

B $v *$

C $v + d * d$

D $(v * d) + d *$

提交

小测试

对于CFG $G_{exp} = (\{E,O\}, \{ (,), +, *, v, d \}, P, E)$，P为

（1）$E \rightarrow EOE$
（2）$E \rightarrow (E)$
（3）$E \rightarrow v$
（4）$E \rightarrow d$
（5）$O \rightarrow +$
（6）$O \rightarrow *$

以下哪些字符串可以归约为E?

$(v+d)*(d+d) \xrightarrow{(3)} (E+d)*(d+d) \xrightarrow{(5)}$

$(EOd)*(d+d) \xrightarrow{(4)} (EOE)*(d+d) \xrightarrow{(1)}$

$(E)*(d+d) \xrightarrow{(2)} E*(d+d) \xrightarrow{(6)} EO(d+d)$

$\ldots \xrightarrow{(2)} EOE \xrightarrow{(1)} E$

小测试

对于CFG $G_{exp} = (\{E,O\}, \{ (,), +, *, v, d \}, P, E)$，P为

（1）$E \rightarrow EOE$
（2）$E \rightarrow (E)$
（3）$E \rightarrow v$
（4）$E \rightarrow d$
（5）$O \rightarrow +$
（6）$O \rightarrow *$

以下哪些字符串可以归约为E?

$$v+d*d \xrightarrow{(3)} E+d*d \xrightarrow{(5)} EOd*d \xrightarrow{(4)} EOE*d$$

$$\xrightarrow{(1)} E*d \xrightarrow{(6)} EOd \xrightarrow{(4)} EOE \xrightarrow{(1)} E$$

派生和归约的形式定义

- 定义:若上下文无关文法 $G = (V, T, P, S)$，设 $\alpha, \beta, \gamma \in (V \cup T)^*$，$A \in V, A \to \gamma \in P$，那么称在$G$中由 $\alpha A \beta$ 可派生出 $\alpha\gamma\beta$，记为

$$\alpha A \beta \underset{G}{\Rightarrow} \alpha\gamma\beta.$$

- 相应的，称 $\alpha\gamma\beta$ 可归约为$\alpha A \beta$.
- 派生$\alpha A \beta \underset{G}{\Rightarrow} \alpha\gamma\beta$，即是用产生式$A \to \gamma$ 的右部γ替换串$\alpha A \beta$中变元A得到串$\alpha\gamma\beta$的过程。
- 如果语境中 G 是已知的，可省略，记为$\alpha A \beta \Rightarrow \alpha\gamma\beta$

派生和归约的形式定义

- 设$\alpha_1, \cdots, \alpha_m \in (V \cup T)^*$，$m \geq 1$，对$i = 1, \cdots, m-1$, 如果有

$$\alpha_i \underset{G}{\Rightarrow} \alpha_{i+1}$$

成立，即α_1通过零步或者多步派生可得到α_m

$$\alpha_1 \underset{G}{\Rightarrow} \alpha_2 \underset{G}{\Rightarrow} \cdots \underset{G}{\Rightarrow} \alpha_m$$

那么记为$\alpha_1 \overset{*}{\underset{G}{\Rightarrow}} \alpha_m$。

- 若α派生出β刚好经过了i步，可记为$\alpha \overset{i}{\underset{G}{\Rightarrow}} \beta$。

派生和归约的形式定义

- 例：算数表达式 $a * (a + b00)$ 在文法G_{exp}中的派生过程.

$$E \Rightarrow E * E \Rightarrow E * (E) \Rightarrow I * (E)$$

$$\Rightarrow I * (E + E) \Rightarrow I * (E + I) \Rightarrow I * (I + I)$$

$$\Rightarrow I * (a + I) \Rightarrow a * (a + I) \Rightarrow a * (a + I0)$$

$$\Rightarrow a * (a + I00) \Rightarrow a * (a + b00)$$

主要内容

- **上下文无关文法**
  - 形式定义
  - 归约和派生
  - 最左派生和最右派生
  - 文法的语言
- **语法分析树**
- **文法和语言的歧义性**
- **文法的化简与范式**

最左派生和最右派生

- 为限制派生的随意性，要求只替换符号串中最左边变元的派生过程，称为最左派生，记为

$$\underset{lm}{\Rightarrow},\quad \overset{*}{\underset{lm}{\Rightarrow}}$$

只替换最右的，称为最右派生，记为

$$\underset{rm}{\Rightarrow},\quad \overset{*}{\underset{rm}{\Rightarrow}}$$

- 任何派生都有等价的最左派生和最右派生。

$A \overset{*}{\Rightarrow} w$当且仅当$A \overset{*}{\underset{lm}{\Rightarrow}} w$，当且仅当$A \overset{*}{\underset{rm}{\Rightarrow}} w$。

最左派生和最右派生

- 例：表达式 $a * (a + a)$ 在G_{exp}中的最左派生为：

1. $E \to I$
2. $E \to E + E$
3. $E \to E * E$
4. $E \to (E)$
5. $I \to a$
6. $I \to b$
7. $I \to Ia$
8. $I \to Ib$
9. $I \to I0$
10. $I \to I1$

$$
\begin{aligned}
E &\underset{\text{lm}}{\Rightarrow} E * E \\
&\underset{\text{lm}}{\Rightarrow} I * E \\
&\underset{\text{lm}}{\Rightarrow} a * E \\
&\underset{\text{lm}}{\Rightarrow} a * (E) \\
&\underset{\text{lm}}{\Rightarrow} a * (E + E) \\
&\underset{\text{lm}}{\Rightarrow} a * (I + E) \\
&\underset{\text{lm}}{\Rightarrow} a * (a + E) \\
&\underset{\text{lm}}{\Rightarrow} a * (a + I) \\
&\underset{\text{lm}}{\Rightarrow} a * (a + a)
\end{aligned}
$$

最左派生和最右派生

- 例：表达式 $a * (a + a)$ 在G_{exp}中的最右派生为：

1. $E \to I$
2. $E \to E + E$
3. $E \to E * E$
4. $E \to (E)$
5. $I \to a$
6. $I \to b$
7. $I \to Ia$
8. $I \to Ib$
9. $I \to I0$
10. $I \to I1$

$$
\begin{aligned}
E &\underset{\text{rm}}{\Rightarrow} E * E \\
&\underset{\text{rm}}{\Rightarrow} E * (E) \\
&\underset{\text{rm}}{\Rightarrow} E * (E + E) \\
&\underset{\text{rm}}{\Rightarrow} E * (E + I) \\
&\underset{\text{rm}}{\Rightarrow} E * (E + a) \\
&\underset{\text{rm}}{\Rightarrow} E * (I + a) \\
&\underset{\text{rm}}{\Rightarrow} E * (a + a) \\
&\underset{\text{rm}}{\Rightarrow} I * (a + a) \\
&\underset{\text{rm}}{\Rightarrow} a * (a + a)
\end{aligned}
$$

最左派生与最右派生

例1：给定上下文无关文法

$$S \to A1B$$
$$A \to 0A \mid \epsilon$$
$$B \to 0B \mid 1B \mid \epsilon$$

给出串 00101 的最右派生。

解：最右派生如下

步骤	产生式	字符串
0		S
1	$S \to A1B$	$A1B$
2	$B \to 0B$	$A10B$
3	$B \to 1B$	$A101B$
4	$B \to \epsilon$	$A101$

最左派生与最右派生

例1：给定上下文无关文法

$$S \to A1B$$
$$A \to 0A \mid \epsilon$$
$$B \to 0B \mid 1B \mid \epsilon$$

给出串 00101 的最右派生。

解：最右派生如下

步骤	产生式	字符串
5	$A \to 0A$	$0A101$
6	$A \to 0A$	$00A101$
7	$A \to \epsilon$	00101

文法 $S \to A1B$
$A \to 0A \mid \epsilon$
$B \to 0B \mid 1B \mid \epsilon$
的00101的最左派生为?

A $S \underset{\text{rm}}{\Rightarrow} A1B \underset{\text{rm}}{\Rightarrow} 0A1B \underset{\text{rm}}{\Rightarrow} 00A1B \underset{\text{rm}}{\Rightarrow} 001B \underset{\text{rm}}{\Rightarrow} 0010B \underset{\text{rm}}{\Rightarrow} 00101B \underset{\text{rm}}{\Rightarrow} 00101$

B $S \underset{\text{rm}}{\Rightarrow} A1B \underset{\text{rm}}{\Rightarrow} A10B \underset{\text{rm}}{\Rightarrow} A101B \underset{\text{rm}}{\Rightarrow} A101 \underset{\text{rm}}{\Rightarrow} 0A10 \underset{\text{rm}}{\Rightarrow} 00A101 \underset{\text{rm}}{\Rightarrow} 00101$

C $S \underset{\text{lm}}{\Rightarrow} A1B \underset{\text{lm}}{\Rightarrow} 0A1B \underset{\text{lm}}{\Rightarrow} 00A1B \underset{\text{lm}}{\Rightarrow} 001B \underset{\text{lm}}{\Rightarrow} 0010B \underset{\text{lm}}{\Rightarrow} 00101B \underset{\text{lm}}{\Rightarrow} 00101$

D $S \underset{\text{lm}}{\Rightarrow} 0A \underset{\text{lm}}{\Rightarrow} 00A \underset{\text{lm}}{\Rightarrow} 00A1B \underset{\text{lm}}{\Rightarrow} 001B \underset{\text{lm}}{\Rightarrow} 0010B \underset{\text{lm}}{\Rightarrow} 00101B \underset{\text{lm}}{\Rightarrow} 00101$

提交

主要内容

- **上下文无关文法**
  - 形式定义
  - 归约和派生
  - 最左派生和最右派生
  - 文法的语言
- **语法分析树**
- **文法和语言的歧义性**
- **文法的化简与范式**

文法的语言

- 定义：上下文无关文法$G=(V,T,P,S)$的语言定义为

$$\boldsymbol{L}(G)=\{w|w\in T^*,S\overset{*}{\underset{G}{\Rightarrow}}w\}$$

即符号串w在$\boldsymbol{L}(G)$中要满足：

1. w仅由终结符组成；
2. 初始符号S能派生出w.

上下文无关语言

- 如果语言L是某个 CFG G 定义的语言，即 $L = L(G)$，则称L为上下文无关语言（CFL，Context-Free Language）
- 上下文无关是指在文法派生的每一步

$$\alpha A\beta \Rightarrow \alpha\gamma\beta$$

符号串γ仅根据A的产生式派生，而无需依赖 A 的上下文 α 和 β.

文法的等价性

- **定义：如果有两个文法 CFG G_1和CFG G_2，满足$\mathbf{L}(G_1) = \mathbf{L}(G_2)$，则称 G_1 和 G_2 是等价的.**

句型

- 若文法$G = (V, T, P, S)$, 初始符号S派生出来的符号串，称为 G 的句型，即$\alpha \in (V \cup T)^*$ 且 $S \overset{*}{\Rightarrow} \alpha$

如果 $S \underset{lm}{\overset{*}{\Rightarrow}} \alpha$，称$\alpha$为左句型

如果 $S \underset{rm}{\overset{*}{\Rightarrow}} \alpha$，称$\alpha$为右句型

- 只含有**终结符**的句型，也称为G的句子
- 而 $\mathbf{L}(G)$ 就是文法G全部的**句子**

示例

- **例：描述CFG** $G = (\{S\}, \{a, b\}, \{S \Longrightarrow aSb, S \Longrightarrow ab\}, S)$ **定义的语言?**

观察开始符号S**可以派生出的串：**

$S \Longrightarrow aSb \Longrightarrow a^2Sb^2 \Longrightarrow \cdots \Longrightarrow a^{n-1}Sb^{n-1}$

利用第二条产生式，有：

$$a^n b^n$$

故$\mathbf{L}(G) = \{a^n b^n | n \geq 1\}$

单选题 25分

设置

由文法S->0S0S1 | 0S1S0 | 1S0S0 | ε， 无法产生下面哪个字符串?

A 001010100

B 110001000

C 100001

D 010010

E 以上都错

提交

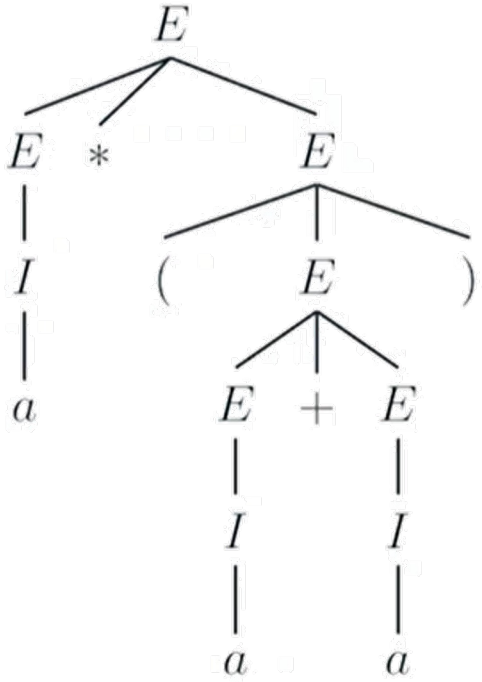
主要内容

- 上下文无关文法
- **语法分析树**
  - 形式定义
  - 语法树和派生的等价性

语法分析树

- 语法分析树用来表示派生（或递归推理），可以从树中看出整个派生过程和最终产生的符号串.
- 例：算数表达式文法G_{exp}派生算数表达式$a*(a+a)$的过程。

1. $E \to I$
2. $E \to E + E$
3. $E \to E * E$
4. $E \to (E)$
5. $I \to a$
6. $I \to b$
7. $I \to Ia$
8. $I \to Ib$
9. $I \to I0$
10. $I \to I1$



语法分析树的形式定义

- **定义：CFG $G = (V, T, P, S)$ 的语法分析树（parse tree），也称为派生树（derivation tree），其形式定义为：**
  - 每个内节点的标记是V中的变元符号
  - 每个叶节点的标记是$V \cup T \cup \{\varepsilon\}$中的符号
  - 如果内节点的标记是A, 其子节点从左至右分别为

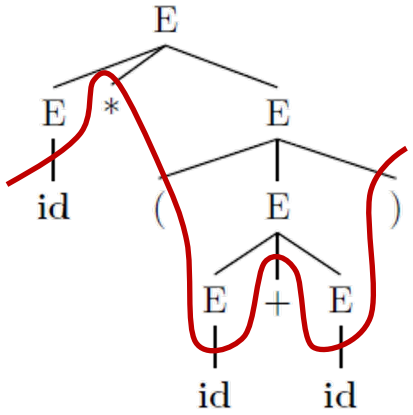
$$X_1, X_2, \ldots, X_n$$

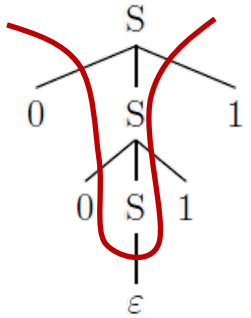
那么，那么，$A \to X_1 X_2 \ldots X_n$是P的一个产生式;

若某个X_i为ε，则X_i一定是A唯一的子节点，且$A \to \varepsilon$是一个产生式。

语法分析树的形式定义

- 定义：语法分析树的全部叶节点从左到右连接起来，称为该树的产物（yield）或结果。如果树根节点是初始符号S，叶节点是终结符，那么该树的产物属于$\mathbf{L}(G)$。
- 定义：语法分析树中标记为A的内节点及其全部子孙节点构成的子树，称为A子树。





语法分析树和派生的等价性

- **定理：CFG** $G = (V, T, P, S)$**且**$A \in V$**，那么文法** G **中**

$$A \overset{*}{\Rightarrow} \alpha$$

当且仅当在文法G**中存在以**A**为根节点产物为**α**的语法分析树。**

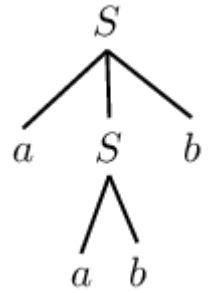
唯一最左（右）派生

- **每棵语法分析树都有唯一的最左（右）派生**
- **给定CFG $G=(V,T,P,S), A\in V$，以下命题等价:**
  - 通过递归推理，确定串w在变元A的语言中
  - 存在以A为根节点，产物为w的语法分析树
  - $A \overset{*}{\Rightarrow} w$
  - $A \underset{lm}{\overset{*}{\Rightarrow}} w$
  - $A \underset{rm}{\overset{*}{\Rightarrow}} w$

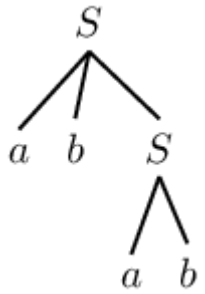
设置

若文法为S -> aSbS | bSaS |ε， 那么字符串aabb的语法分析树为

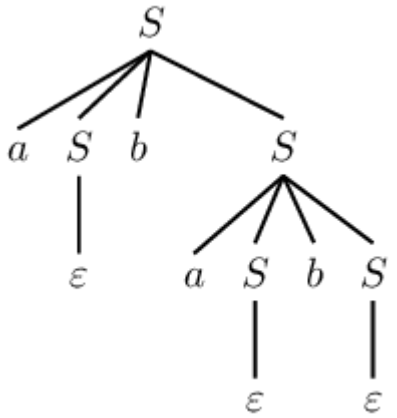
A



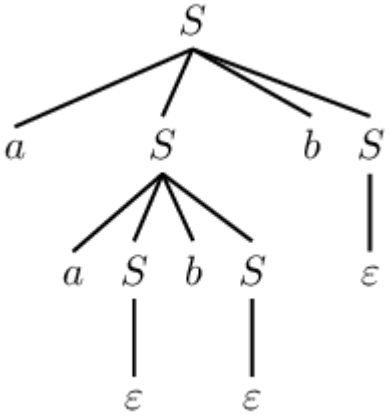
B



C



D



提交

例2：请给出下列语言的一个上下文无关文法

$$L = \{0^n 1^n | n \geq 1\}$$

解：该语言具有递归的特点，所以可以先写出递归式

$$S \to 0S1$$

由于上式是一个无限递归，必须要增加基础的情况

$$S \to 01$$

这样也满足了 $n \geq 1$ 的条件。因此最终的文法如下

$$S \to 0S1 \mid 01$$

例3：请给出下列语言的一个上下文无关文法

$$L = \{0^n 1^n | n \geq 0\}$$

解：这道题和例1非常像，唯一的区别在于把 $n \geq 1$ 改成了 $n \geq 0$。因此，递归部分不变，只需修改基础部分

$$S \rightarrow 0S1 \mid \epsilon$$

例4：请给出下列语言的一个上下文无关文法

$$L = \{a^m b^n | m \geq 0 \wedge n \geq 0 \wedge 3m \geq n \geq 2m\}$$

解：这道题一眼看上去感觉无从下手，如何实现区间关系？

不管那么多，不妨先想办法实现 $n = 2m$

$$S \rightarrow aSbb$$

显然，这种递归形式可以保证2倍的关系。那么3倍的关系也很容易实现了

$$S \rightarrow aSbbb$$

那么怎么实现区间关系呢？

例4：请给出下列语言的一个上下文无关文法

$$L = \{a^m b^n | m \geq 0 \wedge n \geq 0 \wedge 3m \geq n \geq 2m\}$$

解：幸运的是，当文法同时包含这两条规则是，恰好实现了区间关系

$$S \rightarrow aSbb$$

$$S \rightarrow aSbbb$$

这是因为两条规则的混用恰好能实现区间关系。加入基础部分，就得到了最终的文法

$$S \rightarrow aSbb \mid aSbbb \mid \epsilon$$

例5：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) = \text{count}(w, b)\}$$

解：在前面的例子里，a 和 b 的顺序是指定的，而这道题就难很多，因为对顺序没有要求，只要求数量相等。可以考虑三种情况。第1种情况是空串 $S \to \epsilon$ 。

第2种情况是字符串以 a 开头，则在剩下的字符串中 b 的数量必然比 a 多1个。对剩下的字符串重新计数，当第1次遇到 b 的数量比 a 多1个时停下来，可将剩下的字符串分成三部分：左部和右部中 a 的数量和 b 相等，中间是一个单独的分割点 b 。

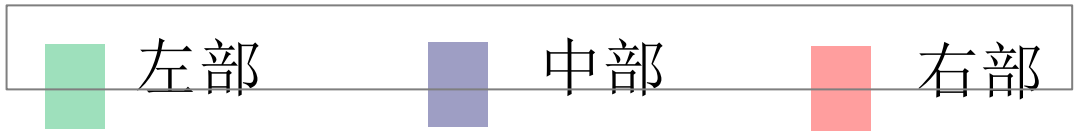
上下文无关文法设计

例5：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a,b\}^*, \text{count}(w,a) = \text{count}(w,b)\}$$

解：不妨看几个具体的例子

$aabbab$　　$ababab$　　$aaaabbbb$



注意：中间部分作为分割点的 b 是唯一的。

因此，很容易用递归的形式来定义

$$S \to aSbS$$

上下文无关文法设计

例5：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) = \text{count}(w, b)\}$$

解：第3种情况是字符串以b 开头，可以类似地递归定义

$$S \to bSaS$$

综上所述，该文法定义为

$$S \to aSbS \mid bSaS \mid \epsilon$$

例 6：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, |\text{count}(w, a) - \text{count}(w, b)| = 2\}$$

解：由于使用了绝对值，因此要区分两种情况

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) - \text{count}(w, b) = 2\} \cup$$
$$\{w | w \in \{a, b\}^*, \text{count}(w, b) - \text{count}(w, a) = 2\}$$

首先考虑第一种情况，即 a 的数量比 b 多2个。依然可以采用之前的分割技术。从左往右扫描，计算 a 的数量和 b 的数量，当 a 的数量第1次比 b 的数量多1时，将该 a 确定为第一个分割点。

上下文无关文法设计

例 6：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, |\text{count}(w, a) - \text{count}(w, b)| = 2\}$$

解：然后将计数清零，类似地去找第2个分割点 a 。这两个分割点可以将字符串切成5个部分

$$bbaa|a|bbaa|a|ba$$

蓝色部分是切割点，红色部分两个元素的个数相等。

因此可以写成：

$$A \to CaCaC$$
$$C \to aCbC \mid bCaC \mid \epsilon$$

例 6：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, |\text{count}(w, a) - \text{count}(w, b)| = 2\}$$

解：第二种情况是对称的，可以类似地处理。

因此，最终的文法是

$$S \to A \mid B$$
$$A \to CaCaC$$
$$B \to CbCbC$$
$$C \to aCbC \mid bCaC \mid \epsilon$$

上下文无关文法设计

例 7：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

解：可以用前面的策略，先把文法拆成两部分

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) > \text{count}(w, b)\} \cup$$
$$\{w | w \in \{a, b\}^*, \text{count}(w, a) < \text{count}(w, b)\}$$

首先考虑第一部分，即 a 的数量比 b 多。该怎么下手？

随机列几个符合条件的字符串，仔细观察

$$aab \qquad aabaaba \qquad baaab \qquad abaaaaba$$

上下文无关文法设计

例 7：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

解：假设对于每个字符串从左往右扫描，边扫描边数 a 和 b 的数量，遇到第一次 a 的数量比 b 多1就停下来。这样可以把字符串分成三部分，左边的部分 a 的数量和 b 相等，中间部分只有1个 a，右边的部分 a 的数量不低于 b

aab　　$aabaaba$　　$baaab$　　$abaaaaba$

注意：中间部分作为分割点的 a 是唯一的。

上下文无关文法设计

例 7：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

解：如何实现 a 的数量不低于 b？可以在数量相等的基础上增加 a 的数量。

$$D \to aDbD \mid bDaD \mid aD \mid Da \mid \epsilon$$

因此，第一种情况可以表示为

$$A \to CaD$$
$$C \to aCbC \mid bCaC \mid \epsilon$$
$$D \to aDbD \mid bDaD \mid aD \mid Da \mid \epsilon$$

上下文无关文法设计

例 7：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

解：第二种情况是对称的，可以类似地处理。

因此，最终的文法是

$$S \to A \mid B$$
$$A \to CaD$$
$$B \to CbE$$
$$C \to aCbC \mid bCaC \mid \epsilon$$
$$D \to aDbD \mid bDaD \mid aD \mid Da \mid \epsilon$$
$$E \to aEbE \mid bEaE \mid bE \mid Eb \mid \epsilon$$

上下文无关文法设计

例 7：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

解：但是上面的答案比较冗余，尤其是后三个式子有些重复。

能不能写得更简单一些？以第一种情况为例，只要分割点两侧都满足 a 的数量不低于 b 即可。

$$abaaaaba$$

这样可以简化一些

$$A \to CaC$$

$$C \to aCbC \mid bCaC \mid aC \mid Ca \mid \epsilon$$

上下文无关文法设计

例 7：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

解：按照这种方式得到的文法比之前少一个式子

$$S \to A \mid B$$
$$A \to CaC$$
$$B \to DbD$$
$$C \to aCbC \mid bCaC \mid aC \mid Ca \mid \epsilon$$
$$D \to aDbD \mid bDaD \mid bD \mid Db \mid \epsilon$$

上下文无关文法设计

例 7：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

解：不过这种方法的分割点不是唯一的。以下分割都合法。

$$a|baaaaba$$

$$ab|aaaaba$$

$$aba|aaaba$$

$$abaa|aaba$$

$$abaaaa|ba$$

$$abaaaaba|$$